```
8
    loss.backward()
9
    with torch.no_grad():
10
        w1 -= lr * w1.grad
11
        w2 -= lr * w2.grad
```

- ***1*** Downloads and loads the MNIST dataset
- ***2*** Sets up a loss function
- ***3*** Gets a set of random index values
- ***4*** Subsets the data and flattens the 28 x 28 images into 784 vectors
- ***5*** Normalizes the vector to be between 0 and 1
- ***6*** Makes a prediction using the neural network
- ***7*** Gets the ground-truth image labels
- ***8*** Computes the loss
- ***9*** Backpropagates
- ***10*** Does not compute gradients in this block
- ***11*** Gradient descent over the parameter matrices

You can tell that the neural network is successfully training by observing the loss function fairly steadily decreasing over training time (figure A.3). This short code snippet trains a complete neural network to successfully classify MNIST digits at around 70% accuracy. We just implemented gradient descent exactly the same way we did with our simple logarithmic function $f(x) = \log(x^4 + x^3 + 2)$, but PyTorch handled the gradients for us. Since the gradient of the neural network's parameters depends on the input data, each time we run the neural network "forward" with a new random sample of images, the gradients will be different. So we run the neural network forward with a random sample of data, PyTorch keeps track of the computations that occur, and when we're done, we call the `backward()` method on the last output; in this case it is generally the loss. The `backward()` method uses automatic differentiation to compute all gradients for all PyTorch variables that have `requires_grad=True` set. Then we can update the model parameters using gradient descent. We wrap the actual gradient descent part in the `torch.no_grad()` context because we don't want it to keep track of these computations.